Entrega 1: Diseño y Configuración del Sistema Híbrido Relacional-Vectorial para RAG

Proyecto Cinepailas

Campo	Información del grupo
Nombre del proyecto	Cinepailas
Dominio de datos	Cine: gestión de cartelera, películas, funciones, salas, boletería y reseñas, con búsqueda semántica sobre el contenido de las películas
Modalidad	Terna
Entidades mínimas exigidas	15
Entidades en el ERD entregado	15 (más 2 tablas intermedias: Pelicula_Genero y Pelicula_Actor)
Integrante 1	Nicolas Olarte C
Integrante 2	Santiago Betancur
Integrante 3	Estith Jakobo Álvarez Quintero
Repositorio Git	https://github.com/santibetan947/bases-de-datos-relacionales
Entorno elegido	Neon (PostgreSQL serverless) + pgvector
Fecha de entrega	14 de octubre de 2026

1. Universo del discurso y análisis de requerimientos

1.1 Descripción del dominio

Cinepailas es un cine que necesita organizar la información de su cartelera: las películas que proyecta, las salas, las funciones con sus horarios, la venta de boletas a sus clientes y las reseñas que estos dejan sobre las películas. Hoy gran parte de esa información son textos libres, como sinopsis y reseñas, y buscar en ellos por palabras exactas no permite encontrar lo que el cliente realmente quiere ver.

El sistema guarda los datos de cartelera y ventas en tablas relacionales, y las sinopsis y reseñas como embeddings en el mismo motor de base de datos. Con eso, un asistente basado en RAG responde preguntas en lenguaje natural, por ejemplo "una película de comedia para ver en familia", usando búsqueda semántica sobre los textos y filtros exactos sobre horarios y salas. Lo usan los clientes que quieren decidir qué película ver y el personal de taquilla que debe resolver dudas rápido.

1.2 Actores y fuentes de datos

Actor / fuente	Descripción	Volumen estimado
Usuario (rol cliente)	Consulta en lenguaje natural qué película ver (por género, tema, duración o tipo de público) y revisa horarios disponibles. También deja reseñas de las películas que vio.	100 usuarios
Usuario (rol taquilla)	Consulta el asistente para resolver dudas de los clientes sobre películas y funciones, y registra la venta de boletas.	5 usuarios
Fuente de documentos	Sinopsis de las películas, más las reseñas de los clientes. Origen: datos de ejemplo cargados por el grupo desde fuentes públicas como TMDB. Idioma: español. Formato: CSV.	240 documentos (40 sinopsis y 200 reseñas)

Actor / fuente	Descripción	Volumen estimado
Fuente de imágenes	Afiches de las películas. Origen: sitios públicos como TMDB, de uso académico. Cada afiche pertenece a una película, la misma de su sinopsis.	40 afiches (uno por película)

1.3 Requerimientos funcionales

ID	Requerimiento	Tipo de consulta que lo satisface
RF-01	Buscar películas por idioma y rango de fechas de estreno	SQL clásica
RF-02	Encontrar sinopsis y reseñas semánticamente similares a una pregunta	Vectorial texto-texto
RF-03	Encontrar afiches de películas a partir de una descripción en texto	Vectorial texto→imagen
RF-04	Combinar filtros de metadatos (género, clasificación, horario de la función) con similitud semántica	Híbrida
RF-05	Generar una respuesta en lenguaje natural recomendando películas con el contexto recuperado	RAG
RF-06	Registrar y consultar métricas de calidad por consulta (tiempo de respuesta y relevancia de los resultados)	SQL analítica
RF-07	Consultar las funciones disponibles por sala y registrar la venta de boletas	SQL clásica

1.4 Requerimientos no funcionales

- **Rendimiento:** las búsquedas vectoriales sobre el dataset (unos 240 documentos y 40 imágenes) responden en menos de 1 segundo.
- **Integridad:** todas las tablas tienen llave primaria (PK); las relaciones entre tablas se garantizan con llaves foráneas (FK), por ejemplo función → película y sala, o boleta → función, usuario y método de pago; se usan restricciones CHECK (precio mayor que 0, calificación de las reseñas entre 1 y 5) y NOT NULL en los campos obligatorios; cada resultado apunta a un documento o a una imagen, nunca a ambos.
- **Reproducibilidad:** la base de datos en Neon se crea ejecutando un solo script, schema.sql (tablas, restricciones, extensión pgvector e índices vectoriales), sobre una base vacía. Los datos de ejemplo y los vectores se cargan después con un script de carga guardado en el mismo repositorio.

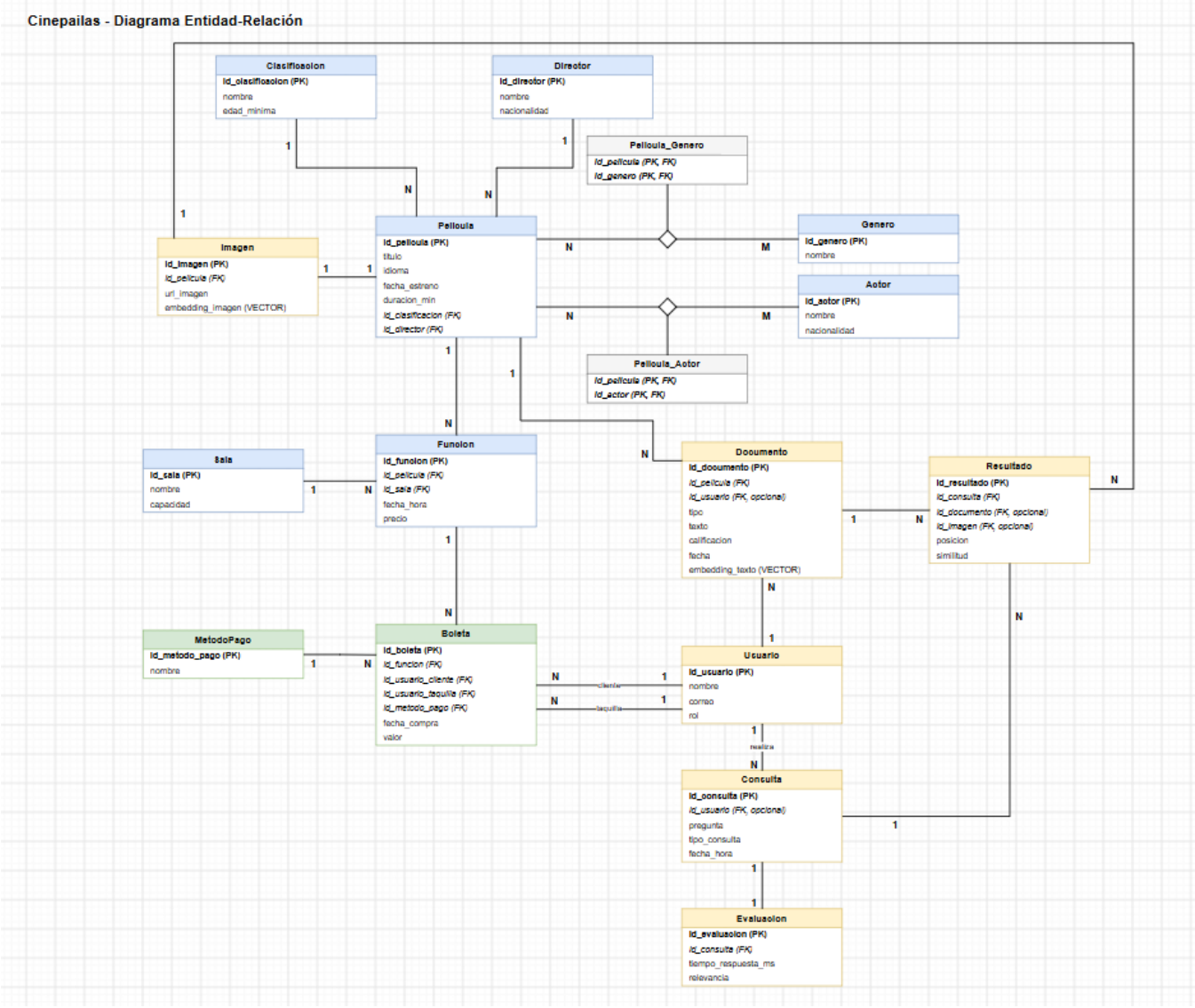
1.5 Supuestos y restricciones del dominio

- Cinepailas es un solo cine; no se modelan varias sedes.
- Una película tiene una única clasificación de edad, y una clasificación puede aplicarse a muchas películas.
- Una película tiene un solo director, y un director puede dirigir varias películas.
- Una película puede tener varios géneros y un género agrupa varias películas.
- Una película tiene varios actores, y un actor puede participar en varias películas.
- Una imagen (afiche) pertenece a una única película, y cada película tiene un solo afiche.
- Una función se proyecta en una sola sala y es de una sola película; una sala puede tener muchas funciones, pero no dos funciones en la misma sala a la misma hora.

- Un usuario con rol de cliente puede comprar muchas boletas; cada boleta pertenece a un solo cliente y a una sola función.
- Cada boleta se paga con un único método de pago (efectivo, tarjeta, etc.) y la registra un usuario con rol de taquilla.
- Las sinopsis y las reseñas se guardan como documentos de una misma entidad, diferenciados por su tipo. Un cliente puede escribir varias reseñas, pero cada reseña es de un solo cliente y sobre una sola película, con una calificación de 1 a 5; la sinopsis no tiene autor ni calificación.
- Un usuario es cliente o personal de taquilla, según su rol; ambos pueden hacer consultas al asistente. Una consulta puede ser anónima (sin usuario).
- Cada consulta devuelve varios resultados (documentos o imágenes) y tiene una única evaluación con su tiempo de respuesta y su relevancia.
- Los textos se vectorizan por fragmentos (chunks) y los afiches como una sola imagen; los vectores se guardan en tablas vectoriales aparte (sección 3).

2. Diseño conceptual (ERD)

2.1 Diagrama entidad-relación



Notación usada: Chen simplificada (cardinalidades 1, N y M sobre las líneas y un rombo en la mitad de cada relación N a M) · Herramienta: draw.io

El diagrama tiene 17 recuadros: las 15 entidades (las 6 del núcleo RAG —Usuario, Documento, Imagen, Consulta, Resultado y Evaluacion— y 9 del dominio) y las 2 tablas intermedias que resuelven las relaciones N a M, Pelicula_Genero y Pelicula_Actor, que no se cuentan entre las 15. Los atributos embedding_texto y embedding_imagen (VECTOR) señalan qué entidades se vectorizan; en el diseño lógico (3.1) esos vectores se guardan en las tablas vectoriales Embeddings_Texto y Embeddings_Imagen, porque un documento puede tener varios fragmentos. La participación de cada relación se detalla en 2.3. Se adjunta el archivo fuente del diagrama (ERD_Cinepailas_actualizado.drawio).

2.2 Catálogo de entidades

Tipo de cada entidad: núcleo, dominio o intermedia.

#	Entidad	Tipo	Descripción breve	Atributo identificador
1	Usuario	Núcleo	Persona que usa el sistema: cliente que consulta y compra boletas, o personal de taquilla (se distingue por el rol).	id_usuario
2	Documento	Núcleo	Texto de una película que se busca: su sinopsis o una reseña de un cliente.	id_documento
3	Imagen	Núcleo	Afiche de una película.	id_imagen
4	Consulta	Núcleo	Pregunta que un usuario le hace al asistente.	id_consulta
5	Resultado	Núcleo	Documento o imagen recuperado para una consulta, con su posición y su similitud.	id_resultado
6	Evaluacion	Núcleo	Tiempo de respuesta y relevancia de una consulta.	id_evaluacion
7	Clasificacion	Dominio	Clasificación de edad de las películas (por ejemplo, todo público o +12).	id_clasificacion
8	Director	Dominio	Director de una o varias películas.	id_director
9	Genero	Dominio	Categoría de película (comedia, terror, drama, etc.).	id_genero
10	Actor	Dominio	Actor que participa en una o varias películas.	id_actor
11	Pelicula	Dominio	Película del cine, con su idioma, fecha de estreno y duración.	id_pelicula
12	Sala	Dominio	Sala donde se proyectan las funciones.	id_sala
13	Funcion	Dominio	Proyección de una película en una sala, a una fecha y hora y con un precio.	id_funcion
14	MetodoPago	Dominio	Forma de pago de una boleta (efectivo, tarjeta, etc.).	id_metodo_pago
15	Boleta	Dominio	Entrada comprada por un cliente para una función y registrada por taquilla.	id_boleta
—	Pelicula_Genero	Intermedia	Une cada película con sus géneros (relación N a M). No cuenta entre las 15.	(id_pelicula, id_genero)
—	Pelicula_Actor	Intermedia	Une cada película con sus actores (relación N a M). No cuenta entre las 15.	(id_pelicula, id_actor)

2.3 Relaciones y cardinalidades

Participación: total significa que toda ocurrencia de la entidad participa en la relación; parcial, que puede no participar. Se indica primero la entidad A y luego la B. La relación "recupera" es N:M y se implementa con Resultado, mediante las tres relaciones 1:N que la siguen.

Relación	Entidad A	Entidad B	Cardinalidad	Participación (total / parcial)	Regla de negocio que la justifica
realiza	Usuario	Consulta	1:N	A parcial B parcial	Un usuario hace varias consultas; una consulta puede ser anónima (sin usuario).

Relación	Entidad A	Entidad B	Cardinalidad	Participación (total / parcial)	Regla de negocio que la justifica
recupera	Consulta	Documento / Imagen	N:M (vía Resultado)	A parcial B parcial	Cada resultado une una consulta con un documento o con una imagen; una consulta puede no tener resultados y un documento o imagen puede no recuperarse nunca.
devuelve	Consulta	Resultado	1:N	A parcial B total	Cada resultado pertenece a una sola consulta, que lo ordena por posición.
es resultado de	Documento	Resultado	1:N	A parcial B parcial	Un documento puede aparecer en resultados de varias consultas; un resultado apunta a un documento solo si no apunta a una imagen.
es resultado de	Imagen	Resultado	1:N	A parcial B parcial	Una imagen puede aparecer en resultados de varias consultas; un resultado apunta a una imagen solo si no apunta a un documento.
se evalúa	Consulta	Evaluacion	1:1	A parcial B total	Cada evaluación mide una sola consulta y cada consulta se evalúa una vez; una consulta puede estar sin evaluar todavía.
clasifica	Clasificacion	Pelicula	1:N	A parcial B total	Una película tiene una única clasificación; una clasificación se aplica a muchas películas.
dirige	Director	Pelicula	1:N	A parcial B total	Una película tiene un solo director; un director puede dirigir varias películas.
pertenece a	Pelicula	Genero (vía Pelicula_Genero)	N:M	A parcial B parcial	Una película puede tener varios géneros y un género agrupa varias películas.
reparto	Pelicula	Actor (vía Pelicula_Actor)	N:M	A parcial B parcial	Una película tiene varios actores y un actor puede participar en varias películas.
tiene	Pelicula	Imagen	1:1	A parcial B total	Cada película tiene un solo afiche y cada afiche es de una sola película.
tiene	Pelicula	Documento	1:N	A parcial B total	Una película tiene varios documentos (su sinopsis y sus reseñas); cada documento es de una sola película.
escribe	Usuario	Documento (reseñas)	1:N	A parcial B parcial	Un cliente escribe varias reseñas; la sinopsis no tiene autor, por eso el usuario es opcional en Documento.
se proyecta en	Pelicula	Funcion	1:N	A parcial B total	Una función es de una sola película; una película puede tener muchas funciones.
aloja	Sala	Funcion	1:N	A parcial B total	Una función va en una sola sala; no hay dos funciones en la misma sala a la misma hora.
genera	Funcion	Boleta	1:N	A parcial B total	Cada boleta es para una sola función; una función puede tener muchas boletas.
compra	Usuario (cliente)	Boleta	1:N	A parcial B total	Un cliente compra varias boletas; cada boleta pertenece a un solo cliente.

Relación	Entidad A	Entidad B	Cardinalidad	Participación (total / parcial)	Regla de negocio que la justifica
registra	Usuario (taquilla)	Boleta	1:N	A parcial B total	Cada boleta la registra un usuario de taquilla, que puede registrar muchas.
paga	MetodoPago	Boleta	1:N	A parcial B total	Cada boleta se paga con un único método de pago.

3. Diseño lógico en 3FN

3.1 Esquema relacional

Una fila por tabla. La PK va subrayada y cada FK indica la tabla que referencia. En el script SQL los nombres de tabla van en minúsculas y con guion bajo (por ejemplo, MetodoPago es metodo_pago).

Tabla	PK	FK (→ tabla referenciada)	Atributos	Restricciones relevantes (UNIQUE, CHECK, NOT NULL)
Usuario	id_usuario	—	nombre, correo, rol	correo UNIQUE; nombre, correo y rol NOT NULL; rol CHECK ('cliente', 'taquilla')
Documento	id_documento	id_pelicula → Pelicula; id_usuario → Usuario (opcional)	tipo, texto, calificacion, fecha	tipo CHECK ('sinopsis', 'resena'); texto y fecha NOT NULL; calificacion CHECK entre 1 y 5; CHECK: si tipo = 'resena', id_usuario y calificacion NOT NULL; si tipo = 'sinopsis', ambos nulos
Imagen	id_imagen	id_pelicula → Pelicula	url_imagen	id_pelicula UNIQUE (1:1 con Pelicula) y NOT NULL; url_imagen NOT NULL
Consulta	id_consulta	id_usuario → Usuario (opcional)	pregunta, tipo_consulta, fecha_hora	pregunta, tipo_consulta y fecha_hora NOT NULL; tipo_consulta CHECK ('sql', 'vectorial', 'hibrida', 'rag')
Resultado	id_resultado	id_consulta → Consulta; id_documento → Documento (opcional); id_imagen → Imagen (opcional)	posicion, similitud	CHECK: exactamente uno de id_documento / id_imagen no nulo; posicion NOT NULL y CHECK > 0; UNIQUE (id_consulta, posicion); similitud NOT NULL y CHECK entre -1 y 1
Evaluacion	id_evaluacion	id_consulta → Consulta	tiempo_respuesta_ms, relevancia	id_consulta UNIQUE (1:1); tiempo_respuesta_ms CHECK ≥ 0; relevancia CHECK entre 0 y 1
Clasificacion	id_clasificacion	—	nombre, edad_minima	nombre UNIQUE NOT NULL; edad_minima CHECK ≥ 0
Director	id_director	—	nombre, nacionalidad	nombre NOT NULL
Genero	id_genero	—	nombre	nombre UNIQUE NOT NULL
Actor	id_actor	—	nombre, nacionalidad	nombre NOT NULL
Pelicula	id_pelicula	id_clasificacion → Clasificacion; id_director → Director	titulo, idioma, fecha_estreno, duracion_min	titulo, idioma y fecha_estreno NOT NULL; duracion_min CHECK > 0; ambas FK NOT NULL

Tabla	PK	FK (→ tabla referenciada)	Atributos	Restricciones relevantes (UNIQUE, CHECK, NOT NULL)
Pelicula_Genero	(id_pelicula, id_genero)	id_pelicula → Pelicula; id_genero → Genero	—	PK compuesta: una película no repite género
Pelicula_Actor	(id_pelicula, id_actor)	id_pelicula → Pelicula; id_actor → Actor	—	PK compuesta: una película no repite actor
Sala	id_sala	—	nombre, capacidad	nombre UNIQUE NOT NULL; capacidad CHECK > 0
Funcion	id_funcion	id_pelicula → Pelicula; id_sala → Sala	fecha_hora, precio	UNIQUE (id_sala, fecha_hora): no hay dos funciones en la misma sala a la misma hora; precio CHECK > 0
MetodoPago	id_metodo_pago	—	nombre	nombre UNIQUE NOT NULL
Boleta	id_boleta	id_funcion → Funcion; id_usuario_cliente → Usuario; id_usuario_taquilla → Usuario; id_metodo_pago → MetodoPago	fecha_compra, valor	todas las FK NOT NULL; valor CHECK > 0
Embeddings_Texto	id_embedding	id_documento → Documento (ON DELETE CASCADE)	chunk_id, estrategia_chunking, chunk_texto, vector_embedding VECTOR(384), modelo	todos NOT NULL; estrategia_chunking CHECK ('fixed-size', 'sentence-aware', 'semantic'); UNIQUE (id_documento, estrategia_chunking, chunk_id)
Embeddings_Imagen	id_embedding	id_imagen → Imagen (ON DELETE CASCADE)	vector_embedding VECTOR(512), modelo	todos NOT NULL

Embeddings_Texto y Embeddings_Imagen son las tablas vectoriales: guardan los vectores que en el ERD aparecen como los atributos embedding_texto y embedding_imagen. En total son 19 tablas: 15 entidades, 2 intermedias y 2 vectoriales.

3.2 Justificación de la normalización

- **1FN:** Todas las tablas tienen llave primaria y cada atributo guarda un único valor atómico. Los dos atributos multivaluados del dominio, los géneros y los actores de una película, no se guardaron como listas dentro de Pelicula: se resolvieron con las tablas intermedias Pelicula_Genero y Pelicula_Actor, con una fila por cada pareja película–género y película–actor. Un documento tiene varios fragmentos y, por lo tanto, varios vectores: no se guardan como una lista dentro de Documento, sino con una fila por fragmento en Embeddings_Texto (y un vector por imagen en Embeddings_Imagen). La entidad Imagen no tiene un atributo de etiquetas, así que no hay multivaluado en ella; si se agregaran etiquetas, se resolverían de la misma forma, con las tablas Etiqueta e Imagen_Etiqueta.
- **2FN:** Solo Pelicula_Genero (id_pelicula, id_genero) y Pelicula_Actor (id_pelicula, id_actor) tienen PK compuesta, y no tienen más atributos que la propia clave, por lo que no puede haber dependencias parciales. Las otras 17 tablas tienen PK de una sola columna, así que cumplen 2FN de forma directa: todo atributo depende de la clave completa. Los UNIQUE compuestos, como (id_sala, fecha_hora) en Funcion, (id_consulta, posicion) en Resultado y (id_documento, estrategia_chunking, chunk_id) en Embeddings_Texto, son claves candidatas adicionales y no cambian esto.
- **3FN:** No existen dependencias transitivas: en cada tabla, los atributos que no son clave dependen solo de la PK y no de otro atributo no clave. Ejemplos de dependencias que se eliminaron:

  - En Pelicula, la edad mínima de la clasificación depende de id_clasificacion y no de id_pelicula (id_pelicula → id_clasificacion → edad_minima). Se movió a la tabla Clasificacion, y Pelicula solo guarda la FK.
  - Algo igual pasaba con el director (nombre y nacionalidad dependen de id_director), que ahora está en la tabla Director.
  - En Boleta, el nombre del método de pago depende de id_metodo_pago y no de id_boleta, por eso vive en MetodoPago. Igualmente, la capacidad de la sala depende de id_sala y no de id_funcion, por eso está en Sala.

3.3 Decisiones de diseño

- **Resultado con dos FK opcionales:** Resultado tiene id_documento e id_imagen, ambos opcionales, con un CHECK que exige que exactamente uno no sea nulo. Se eligió esto en lugar de dos tablas separadas (Resultado_Documento y Resultado_Imagen) porque una consulta devuelve una sola lista ordenada que mezcla textos e imágenes: con una tabla, posicion y similitud son comparables y UNIQUE (id_consulta, posicion) garantiza un orden único, y el contexto del RAG se arma con una sola consulta SQL. Con dos tablas habría que duplicar columnas y unir los resultados para reconstruir el orden. A cambio, aparecen valores nulos, que el CHECK controla.
- **Documento unifica sinopsis y reseñas:** Las sinopsis y las reseñas están en una sola tabla con un campo tipo, en lugar de dos tablas. Las dos son texto y se buscan igual en la búsqueda semántica, así que una sola tabla vectorial (Embeddings_Texto) y un solo índice sirven para ambas. A cambio, id_usuario y calificacion quedan nulos en las sinopsis, y un CHECK condicional lo controla.
- **Vectores en tablas separadas:** Los vectores no son columnas de Documento ni de Imagen, sino tablas aparte (Embeddings_Texto y Embeddings_Imagen). Un documento puede tener varios fragmentos, uno por cada estrategia de chunking que se compara en la sección 7, y cada fila guarda el modelo con el que se calculó el vector. Además, los vectores de texto (384) y de imagen (512) tienen dimensiones distintas, y cada tabla tiene su propio índice HNSW. En el ERD se muestran como atributos (VECTOR) solo para señalar qué entidades se vectorizan.
- **Usuario con rol en lugar de Cliente y Empleado:** Los clientes y el personal de taquilla son personas que usan el asistente y comparten los mismos atributos (nombre y correo), así que se modelaron como una sola tabla con el campo rol. Boleta tiene dos FK hacia Usuario, una para el cliente y otra para quien la registra en taquilla. El script no impide asignar como cliente a un usuario de taquilla, o al revés; esa validación de rol se hace al cargar los datos o en la aplicación.
- **Tablas catálogo frente a atributos:** Clasificacion, Genero, MetodoPago, Director, Actor y Sala son tablas porque tienen datos propios (por ejemplo, edad_minima depende de la clasificación) o se repiten en muchas filas (un género o un actor aparece en muchas películas). Así se evitan errores de escritura y un dato se cambia en un solo lugar. En cambio, idioma es un atributo de Pelicula: es un texto corto sin datos propios que dependan de él y se usa como filtro directo en RF-01, por lo que una tabla catálogo agregaría un JOIN sin ganar nada en 3FN. La fuente de los documentos (TMDB) es la misma para todo el conjunto de datos, por eso no se modela como columna ni como tabla.
- **Boleta guarda su propio valor:** Boleta incluye valor aunque Funcion tiene precio, porque valor es lo que realmente se cobró en la venta y el precio de la función puede cambiar después. Así el historial de ventas no se altera, y no es un dato derivado repetido.
- **Evaluacion separada de Consulta:** Las métricas de calidad se calculan después de generar la respuesta. Mantenerlas en Evaluacion, con id_consulta UNIQUE (relación 1 a 1), evita llenar Consulta de columnas que se completan más tarde.

4. Identificación de campos vectorizables

Campo origen	Tabla origen	Modalidad	Modelo de embedding	Dimensión	Tabla destino	Justificación
texto (por chunk)	Documento	Texto	all-MiniLM-L6-v2	384	Embeddings_Texto	Modelo liviano y rápido para texto. Cada documento se divide en chunks según la estrategia de la sección 7 y cada chunk tiene su vector; como las sinopsis y reseñas son cortas, muchos documentos quedan en un solo chunk.
titulo	Pelicula	Texto	—	—	—	Opcional: no se vectoriza; el título se busca con SQL.
pregunta	Consulta	Texto	all-MiniLM-L6-v2	384	No se guarda	Se vectoriza al momento de buscar, con el mismo modelo de los documentos para poder compararlos.
Archivo de imagen (píxeles)	Imagen	Imagen	CLIP ViT-B/32	512	Embeddings_Imagen	CLIP convierte el afiche en un vector que se puede comparar con texto.
descripcion / etiquetas	Imagen	Texto	—	—	—	No aplica: Imagen no tiene descripción ni etiquetas.

- **Imagen:** el vector de imagen se calcula sobre el contenido visual del afiche, que se descarga desde url_imagen, se carga con PIL y se procesa con CLIP; no se vectoriza el texto de la URL.
- **Búsqueda texto→imagen:** la pregunta se convierte en vector con el codificador de texto de CLIP (512) y se compara con embeddings_imagen, que también es de CLIP (512), así los dos vectores quedan en el mismo espacio. MiniLM (384) se usa solo para comparar texto con texto (pregunta contra embeddings_texto); nunca se mezclan vectores de 384 con los de 512.
- **Métrica de distancia elegida:** coseno, porque compara la dirección de los vectores sin importar su tamaño, funciona bien con MiniLM y CLIP, y en pgvector se usa con el operador <=>.

5. Esquema SQL (tablas relacionales y vectoriales)

El esquema completo se encuentra en el archivo schema.sql del repositorio y se ejecuta sin errores sobre una base de datos vacía en Neon (PostgreSQL con pgvector). Crea las 15 entidades del ERD, las tablas intermedias Pelicula_Genero y Pelicula_Actor, y las tablas vectoriales embeddings_texto (VECTOR(384)) y embeddings_imagen (VECTOR(512)) con sus índices HNSW de similitud coseno: 19 tablas en total.

Archivo en el repositorio: schema.sql

```
-- ================================================
-- Cinepailas - Esquema SQL (PostgreSQL + pgvector)
-- Entrega 1: sistema híbrido relacional-vectorial para RAG
-- Se ejecuta sobre una base de datos vacía (Neon)
-- ================================================

CREATE EXTENSION IF NOT EXISTS vector;

-- ---------- Tablas sin dependencias ----------

CREATE TABLE usuario (
    id_usuario SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(100) NOT NULL UNIQUE,
    rol VARCHAR(10) NOT NULL CHECK (rol IN ('cliente', 'taquilla'))
);
```

```
CREATE TABLE clasificacion (
    id clasificacion SERIAL PRIMARY KEY,
    nombre VARCHAR(30) NOT NULL UNIQUE,
    edad minima INT NOT NULL CHECK (edad minima >= 0)
);

CREATE TABLE director (
    id director SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    nacionalidad VARCHAR(50)
);

CREATE TABLE genero (
    id genero SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE actor (
    id_actor SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    nacionalidad VARCHAR(50)
);

CREATE TABLE sala (
    id sala SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    capacidad INT NOT NULL CHECK (capacidad > 0)
);

CREATE TABLE metodo_pago (
    id_metodo_pago SERIAL PRIMARY KEY,
    nombre VARCHAR(30) NOT NULL UNIQUE
);

-- ---------- Películas ----------

CREATE TABLE pelicula (
    id pelicula SERIAL PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    idioma VARCHAR(30) NOT NULL,
    fecha_estreno DATE NOT NULL,
    duracion_min INT NOT NULL CHECK (duracion_min > 0),
    id clasificacion INT NOT NULL REFERENCES clasificacion(id clasificacion),
    id_director INT NOT NULL REFERENCES director(id_director)
);

-- Tablas intermedias (relaciones N a M)
CREATE TABLE pelicula_genero (
    id pelicula INT NOT NULL REFERENCES pelicula(id pelicula),
    id_genero INT NOT NULL REFERENCES genero(id_genero),
    PRIMARY KEY (id pelicula, id genero)
);

CREATE TABLE pelicula_actor (
    id pelicula INT NOT NULL REFERENCES pelicula(id pelicula),
    id_actor INT NOT NULL REFERENCES actor(id_actor),
    PRIMARY KEY (id_pelicula, id_actor)
);

-- Un afiche por película (UNIQUE = relación 1 a 1)
CREATE TABLE imagen (
    id imagen SERIAL PRIMARY KEY,
    id_pelicula INT NOT NULL UNIQUE REFERENCES pelicula(id_pelicula),
    url_imagen TEXT NOT NULL
);

-- Sinopsis y reseñas (tipo = 'sinopsis' o 'resena')
-- id usuario y calificacion solo se llenan en las reseñas
CREATE TABLE documento (
    id_documento SERIAL PRIMARY KEY,
    id_pelicula INT NOT NULL REFERENCES pelicula(id_pelicula),
```

```
    id_usuario INT REFERENCES usuario(id_usuario),
    tipo VARCHAR(10) NOT NULL CHECK (tipo IN ('sinopsis', 'resena')),
    texto TEXT NOT NULL,
    calificacion INT CHECK (calificacion BETWEEN 1 AND 5),
    fecha DATE NOT NULL,
    CHECK (
        (tipo = 'sinopsis' AND id_usuario IS NULL AND calificacion IS NULL) OR
        (tipo = 'resena' AND id usuario IS NOT NULL AND calificacion IS NOT NULL)
    )
);

-- ---------- Funciones y boletas ----------

CREATE TABLE funcion (
    id funcion SERIAL PRIMARY KEY,
    id_pelicula INT NOT NULL REFERENCES pelicula(id_pelicula),
    id_sala INT NOT NULL REFERENCES sala(id_sala),
    fecha_hora TIMESTAMP NOT NULL,
    precio NUMERIC(10,2) NOT NULL CHECK (precio > 0),
    UNIQUE (id_sala, fecha_hora) -- no hay dos funciones en la misma sala a la misma hora
);

CREATE TABLE boleta (
    id_boleta SERIAL PRIMARY KEY,
    id funcion INT NOT NULL REFERENCES funcion(id funcion),
    id_usuario_cliente INT NOT NULL REFERENCES usuario(id_usuario),
    id usuario taquilla INT NOT NULL REFERENCES usuario(id usuario),
    id_metodo_pago INT NOT NULL REFERENCES metodo_pago(id_metodo_pago),
    fecha_compra TIMESTAMP NOT NULL,
    valor NUMERIC(10,2) NOT NULL CHECK (valor > 0)
);

-- ---------- Consultas del asistente ----------

CREATE TABLE consulta (
    id_consulta SERIAL PRIMARY KEY,
    id_usuario INT REFERENCES usuario(id_usuario), -- opcional: la consulta puede ser anónima
    pregunta TEXT NOT NULL,
    tipo_consulta VARCHAR(20) NOT NULL
        CHECK (tipo_consulta IN ('sql', 'vectorial', 'hibrida', 'rag')),
    fecha_hora TIMESTAMP NOT NULL
);

-- Cada resultado es un documento O una imagen (el otro queda en NULL)
CREATE TABLE resultado (
    id_resultado SERIAL PRIMARY KEY,
    id_consulta INT NOT NULL REFERENCES consulta(id_consulta),
    id_documento INT REFERENCES documento(id_documento),
    id imagen INT REFERENCES imagen(id imagen),
    posicion INT NOT NULL CHECK (posicion > 0),
    similitud NUMERIC(6,5) NOT NULL CHECK (similitud BETWEEN -1 AND 1),
    CHECK (
        (id_documento IS NOT NULL AND id_imagen IS NULL) OR
        (id_documento IS NULL AND id_imagen IS NOT NULL)
    ),
    UNIQUE (id consulta, posicion)
);

-- Una evaluación por consulta (UNIQUE = relación 1 a 1)
CREATE TABLE evaluacion (
    id evaluacion SERIAL PRIMARY KEY,
    id_consulta INT NOT NULL UNIQUE REFERENCES consulta(id_consulta),
    tiempo respuesta ms INT NOT NULL CHECK (tiempo respuesta ms >= 0),
    relevancia NUMERIC(3,2) NOT NULL CHECK (relevancia BETWEEN 0 AND 1)
);

-- ====================================================
-- Tablas vectoriales
-- ====================================================

CREATE TABLE embeddings_texto (
    id_embedding SERIAL PRIMARY KEY,
```

```
    id_documento INT NOT NULL REFERENCES documento(id_documento) ON DELETE CASCADE,
    chunk id INT NOT NULL,
    estrategia_chunking VARCHAR(20) NOT NULL
        CHECK (estrategia chunking IN ('fixed-size', 'sentence-aware', 'semantic')),
    chunk_texto TEXT NOT NULL,
    vector_embedding VECTOR(384) NOT NULL,
    modelo VARCHAR(100) NOT NULL,
    UNIQUE (id documento, estrategia chunking, chunk id)
);

CREATE TABLE embeddings_imagen (
    id_embedding SERIAL PRIMARY KEY,
    id_imagen INT NOT NULL REFERENCES imagen(id_imagen) ON DELETE CASCADE,
    vector embedding VECTOR(512) NOT NULL,
    modelo VARCHAR(100) NOT NULL
);

-- Índices vectoriales (distancia coseno)
CREATE INDEX ON embeddings_texto USING hnsw (vector_embedding vector_cosine_ops);
CREATE INDEX ON embeddings_imagen USING hnsw (vector_embedding vector_cosine_ops);
```

6. Entorno ejecutable

Elemento	Valor
Opción	Neon (PostgreSQL serverless) + pgvector
Versión de PostgreSQL	PostgreSQL 16
pgvector	Extensión soportada nativamente por Neon
Lenguaje y librerías de conexión	Python, psycopg2, SQLAlchemy
Archivo de arranque	schema.sql (Ejecutable directamente en el editor SQL de Neon)

7. Plan de experimento de chunking

En este curso se implementan y comparan las tres estrategias. Si modifican los parámetros sugeridos, justifiquen el cambio según su dominio.

7.1 Estrategias y parámetros

Estrategia: Fixed-size

Parámetros sugeridos: chunk_size = 256 tokens, overlap = 32

Parámetros del grupo: chunk_size = 256 tokens, overlap = 32

Herramienta / librería: langchain (TokenTextSplitter / RecursiveCharacterTextSplitter)

Por qué podría funcionar en su dominio: Garantiza fragmentos homogéneos en tamaño y procesamiento uniforme. Es adecuada para sinopsis o reseñas más extensas, asegurando que no se pierda continuidad conceptual entre bloques continuos.

Estrategia: Sentence-aware

Parámetros sugeridos: máx. 5 oraciones, overlap = 1 oración

Parámetros del grupo: máx. 5 oraciones, overlap = 1 oración

Herramienta / librería: spaCy (es_core_news_sm) / NLTK

Por qué podría funcionar en su dominio: Preserva la coherencia sintáctica e idiofónica de las oraciones completas. Evita cortar ideas clave en la descripción de tramas de películas o juicios de valor expresados por los usuarios en las reseñas.

Estrategia: Semantic

Parámetros sugeridos: umbral coseno entre 0.75 y 0.85

Parámetros del grupo: umbral coseno = 0.80

Herramienta / librería: semantic-chunkers / Lógica propia con MiniLM

Por qué podría funcionar en su dominio: Agrupa enunciados según continuidad temática y cambios de sentido. Es ideal para reseñas detalladas donde el usuario transita entre la opinión sobre la actuación, la dirección y la música, o sinopsis que dividen la introducción, nudo y desenlace.

Hipótesis: Se espera que la estrategia Sentence-aware obtenga el mejor desempeño en este dominio. Debido a que la mayoría de los documentos son sinopsis y reseñas cortas (de entre 1 y 5 oraciones), esta técnica conserva la totalidad de la unidad argumental y gramatical sin fragmentar ideas completas ni sobrepasar los límites de contexto de los modelos de embedding empleados.